WEEK 6: PRODUCT REVIEW AND RATING MANAGEMENT SYSTEM

-HEMAVATHI.B (E25AI114)

OBJECTIVE

To design and implement a Product Review and Rating Management System that allows customers to submit reviews and ratings for purchased products, securely stores and manages feedback, and uses SQL queries and aggregate functions to analyze product ratings, identify highly rated products, and generate useful customer feedback reports.

1. Create Review and Rating Tables:

```
SQL> CREATE TABLE Review_Rating (
  2      REVIEW_ID VARCHAR2(10) PRIMARY KEY,
  3      CUSTOMER_ID VARCHAR2(20),
  4      PRODUCT_ID VARCHAR2(10),
  5      RATING NUMBER(1) CHECK (RATING BETWEEN 1 AND 5),
  6      REVIEW_TEXT VARCHAR2(500),
  7      REVIEW_DATE DATE,
  8      FOREIGN KEY (CUSTOMER_ID) REFERENCES CUSTOMER(CUSTOMER_ID),
  9      FOREIGN KEY (PRODUCT_ID) REFERENCES BRAND(PRODUCT_ID)
 10  );

Table created.
```

Store Customer Feedback and Ratings

2. Add reviews for purchased products.-Inserting values into the table

```
SQL> INSERT INTO REVIEW_RATING VALUES ('REV001','C-1001','BR-01',5,'Excellent perfume, long lasting','01-OCT-2026');

1 row created.

SQL> INSERT INTO REVIEW_RATING VALUES ('REV002','C-1002','BR-02',4,'Very good fragrance','01-OCT-2026');

1 row created.

SQL> INSERT INTO REVIEW_RATING VALUES ('REV003','C-1003','BR-03',5,'Amazing product and quality','02-OCT-2026');

1 row created.

SQL> INSERT INTO REVIEW_RATING VALUES ('REV004','C-1004','BR-04',3,'Good but could be better','02-OCT-2026');

1 row created.

SQL> INSERT INTO REVIEW_RATING VALUES ('REV005','C-1005','BR-05',4,'Nice fragrance and packaging','03-OCT-2026');

1 row created.

SQL> INSERT INTO REVIEW_RATING VALUES ('REV006','C-1006','BR-06',5,'Loved the fragrance','03-OCT-2026');

1 row created.

SQL> INSERT INTO REVIEW_RATING VALUES ('REV007','C-1007','BR-07',2,'Fragrance does not last long','04-OCT-2026');

1 row created.

SQL> INSERT INTO REVIEW_RATING VALUES ('REV008','C-1008','BR-08',4,'Worth the price','04-OCT-2026');

1 row created.

SQL> INSERT INTO REVIEW_RATING VALUES ('REV009','C-1009','BR-09',5,'Excellent and refreshing','05-OCT-2026');
```

Displaying the Table after Insertion:

```
SQL> select * from review_rating;

REVIEW_ID  CUSTOMER_ID          PRODUCT_ID     RATING
---------- -------------------- ---------- ----------
REVIEW_TEXT
--------------------------------------------------------------------------------
REVIEW_DA
---------
REV001     C-1001               BR-01               5
Excellent perfume, long lasting
01-OCT-26

REV002     C-1002               BR-02               4
Very good fragrance
01-OCT-26

REVIEW_ID  CUSTOMER_ID          PRODUCT_ID     RATING
---------- -------------------- ---------- ----------
REVIEW_TEXT
--------------------------------------------------------------------------------
REVIEW_DA
---------

REV003     C-1003               BR-03               5
Amazing product and quality
02-OCT-26

REV004     C-1004               BR-04               3
Good but could be better
```

3. Store customer ratings from 1 to 5 stars.

```
SQL> ALTER TABLE REVIEW_RATING ADD CONSTRAINT CHK_RATING CHECK (RATING >= 1 AND RATING <= 5);

Table altered.
```

4. Update review comments if required.

```
SQL> UPDATE REVIEW_RATING SET REVIEW_TEXT = 'Very good fragrance and quality' WHERE REVIEW_ID = 'REV001';

1 row updated.
```

```
SQL> INSERT INTO REVIEW_RATING VALUES ('REV011','C-1002','BR-03',6,'Excellent product','05-OCT-2026');
INSERT INTO REVIEW_RATING VALUES ('REV011','C-1002','BR-03',6,'Excellent product','05-OCT-2026')
*
ERROR at line 1:
ORA-02290: check constraint (C##HEMAVATHI.CHK_RATING) violated
```

5. Remove inappropriate or invalid reviews

```
SQL> DELETE FROM REVIEW_RATING WHERE REVIEW_ID = 'REV007';

1 row deleted.
```

6. Retrieve Product Review Details:

Writing the SQL Queries for the following

1. **Display all reviews for a product.**

```
SQL> SELECT * FROM REVIEW_RATING WHERE PRODUCT_ID = 'BR-01';

REVIEW_ID  CUSTOMER_ID          PRODUCT_ID     RATING
---------- -------------------- ---------- ----------
REVIEW_TEXT
--------------------------------------------------------------------------------
REVIEW_DA
---------
REV001     C-1001               BR-01               5
Very good fragrance and quality
01-OCT-26
```

2. **Display customer name with their reviews.**

```
SQL> SELECT C.FIRST_NAME || ' ' || C.LAST_NAME AS CUSTOMER_NAME, R.REVIEW_TEXT FROM CUSTOMER C JOIN REVIEW_RATING R ON C.CUSTOMER_ID = R.CUSTOMER_ID
;

CUSTOMER_NAME
----------------------------------------
REVIEW_TEXT
--------------------------------------------------------------------------------
Chiru Raju
Very good fragrance and quality

Hema Bhaskar
Very good fragrance

Seetha Mani
Amazing product and quality


CUSTOMER_NAME
----------------------------------------
REVIEW_TEXT
--------------------------------------------------------------------------------
Abinaya Ramesh
Good but could be better

Deepikaa Selvam
Nice fragrance and packaging

Sanjana Doss
Loved the fragrance
```

3. **Find products having maximum reviews.**

```
SQL> SELECT PRODUCT_ID, COUNT(*) AS REVIEW_COUNT FROM REVIEW_RATING GROUP BY PRODUCT_ID HAVING COUNT(*) = (SELECT MAX(COUNT(*)) FROM REVIEW_RATING G
ROUP BY PRODUCT_ID);

PRODUCT_ID REVIEW_COUNT
---------- ------------
BR-01                 1
BR-03                 1
BR-05                 1
BR-06                 1
BR-09                 1
BR-08                 1
BR-04                 1
BR-10                 1
BR-02                 1
BR-07                 1

10 rows selected.
```

4. **Display recent customer feedback.**

```
SQL> SELECT C.FIRST_NAME || ' ' || C.LAST_NAME AS CUSTOMER_NAME, R.REVIEW_TEXT, R.REVIEW_DATE FROM CUSTOMER C JOIN REVIEW_RATING R ON C.CUSTOMER_ID
= R.CUSTOMER_ID ORDER BY R.REVIEW_DATE DESC;

CUSTOMER_NAME
--------------------------------------------------
REVIEW_TEXT
--------------------------------------------------------------------------------
REVIEW_DA
---------
Bhavana Karthik
Excellent and refreshing
05-OCT-26

Chiru Raju
Average product
05-OCT-26

CUSTOMER_NAME
--------------------------------------------------
REVIEW_TEXT
--------------------------------------------------------------------------------
REVIEW_DA
---------

Priya Hari
Worth the price
04-OCT-26

Vanaja Muniyappan
Fragrance does not last long
```

5. **Retrieve reviews with ratings above 4.**

```
SQL> SELECT * FROM REVIEW_RATING WHERE RATING > 4;

REVIEW_ID  CUSTOMER_ID          PRODUCT_ID     RATING
---------- -------------------- ---------- ----------
REVIEW_TEXT
--------------------------------------------------------------------------------
REVIEW_DA
---------
REV001     C-1001               BR-01               5
Excellent perfume, long lasting
01-OCT-26

REV003     C-1003               BR-03               5
Amazing product and quality
02-OCT-26

REVIEW_ID  CUSTOMER_ID          PRODUCT_ID     RATING
---------- -------------------- ---------- ----------
REVIEW_TEXT
--------------------------------------------------------------------------------
REVIEW_DA
---------

REV006     C-1006               BR-06               5
Loved the fragrance
03-OCT-26

REV009     C-1009               BR-09               5
Excellent and refreshing
```

6. Calculate Average Product Ratings Use aggregate functions to analyze ratings

TASKS:

1. Calculate average rating for each product.

```
SQL> SELECT B.BRAND_NAME AS PRODUCT_NAME, AVG(R.RATING) AS AVERAGE_RATING
  2  FROM BRAND B JOIN REVIEW_RATING R ON B.PRODUCT_ID = R.PRODUCT_ID
  3  GROUP BY B.BRAND_NAME;

PRODUCT_NAME                   AVERAGE_RATING
------------------------------ --------------
Chanel                                      4
Armani                                      4
Burberry                                    5
Fogg                                        3
Calvin                                      2
Hugo Boss                                   4
YSL                                         5
Dior                                        5
Gucci                                       5
Versace                                     3

10 rows selected.
```

2. Count the number of reviews per product.

```
SQL> SELECT B.BRAND_NAME AS PRODUCT_NAME, COUNT(R.RATING) AS COUNT_RATING
  2  FROM BRAND B JOIN REVIEW_RATING R ON B.PRODUCT_ID = R.PRODUCT_ID
  3  GROUP BY B.BRAND_NAME;

PRODUCT_NAME                   COUNT_RATING
------------------------------ ------------
Chanel                                    1
Armani                                    1
Burberry                                  1
Fogg                                      1
Calvin                                    1
Hugo Boss                                 1
YSL                                       1
Dior                                      1
Gucci                                     1
Versace                                   1

10 rows selected.
```

3. Find highest-rated products.

```
SQL> SELECT B.PRODUCT_ID, B.BRAND_NAME AS PRODUCT_NAME, AVG(R.RATING) AS AVERAGE_RATING
  2  FROM BRAND B JOIN REVIEW_RATING R ON B.PRODUCT_ID = R.PRODUCT_ID
  3  GROUP BY B.PRODUCT_ID, B.BRAND_NAME
  4  ORDER BY AVERAGE_RATING DESC;

PRODUCT_ID PRODUCT_NAME                   AVERAGE_RATING
---------- ------------------------------ --------------
BR-01      Dior                                        5
BR-09      YSL                                         5
BR-03      Gucci                                       5
BR-06      Burberry                                    5
BR-08      Hugo Boss                                   4
BR-02      Chanel                                      4
BR-05      Armani                                      4
BR-04      Versace                                     3
BR-10      Fogg                                        3
BR-07      Calvin                                      2

10 rows selected.
```

4. Identify products with average rating above 4.

```
SQL> SELECT B.PRODUCT_ID, B.BRAND_NAME AS PRODUCT_NAME, AVG(R.RATING) AS AVERAGE_RATING
  2  FROM BRAND B JOIN REVIEW_RATING R ON B.PRODUCT_ID = R.PRODUCT_ID
  3  GROUP BY B.PRODUCT_ID, B.BRAND_NAME
  4  HAVING AVG(R.RATING) > 4;

PRODUCT_ID PRODUCT_NAME                   AVERAGE_RATING
---------- ------------------------------ --------------
BR-01      Dior                                        5
BR-03      Gucci                                       5
BR-09      YSL                                         5
BR-06      Burberry                                    5
```

7. Identify Highly Rated Products
Generate product rating reports.

Report 1: Product Rating Analysis

Display:

1. **Product name**
2. **Number of reviews**
3. **Average rating**

```
SQL> SELECT B.BRAND_NAME AS PRODUCT_NAME, COUNT(R.REVIEW_ID) AS NUMBER_OF_REVIEWS, AVG(R.RATING) AS AVERAGE_RATING FROM BRAND B JOIN REVIEW_RATING R
 ON B.PRODUCT_ID = R.PRODUCT_ID GROUP BY B.BRAND_NAME;

PRODUCT_NAME                   NUMBER_OF_REVIEWS AVERAGE_RATING
------------------------------ ----------------- --------------
Chanel                                         1              4
Armani                                         1              4
Burberry                                       1              5
Fogg                                           1              3
Calvin                                         1              2
Hugo Boss                                      1              4
YSL                                            1              5
Dior                                           1              5
Gucci                                          1              5
Versace                                        1              3

10 rows selected.
```

Report 2: Customer Feedback Analysis

> Most reviewed products

```
SQL> SELECT B.BRAND_NAME AS PRODUCT_NAME, COUNT(R.REVIEW_ID) AS REVIEW_COUNT FROM BRAND B JOIN REVIEW_RATING R ON B.PRODUCT_ID = R.PRODUCT_ID GROUP
BY B.BRAND_NAME ORDER BY REVIEW_COUNT DESC;

PRODUCT_NAME                   REVIEW_COUNT
------------------------------ ------------
Chanel                                    1
Armani                                    1
Burberry                                  1
Fogg                                      1
Versace                                   1
Hugo Boss                                 1
YSL                                       1
Dior                                      1
Gucci                                     1
Calvin                                    1
```

Highly rated products

```
SQL> SELECT B.BRAND_NAME AS PRODUCT_NAME, AVG(R.RATING) AS AVERAGE_RATING FROM BRAND B JOIN REVIEW_RATING R ON B.PRODUCT_ID = R.PRODUCT_ID GROUP BY
B.BRAND_NAME HAVING AVG(R.RATING) >= 4 ORDER BY AVERAGE_RATING DESC;

PRODUCT_NAME                   AVERAGE_RATING
------------------------------ --------------
Dior                                        5
YSL                                         5
Burberry                                    5
Gucci                                       5
Armani                                      4
Chanel                                      4
Hugo Boss                                   4

7 rows selected.
```

Products requiring improvement

```
SQL> SELECT B.BRAND_NAME AS PRODUCT_NAME, AVG(R.RATING) AS AVERAGE_RATING FROM BRAND B JOIN REVIEW_RATING R ON B.PRODUCT_ID = R.PRODUCT_ID GROUP BY
B.BRAND_NAME HAVING AVG(R.RATING) < 3 ORDER BY AVERAGE_RATING;

PRODUCT_NAME                   AVERAGE_RATING
------------------------------ --------------
Calvin                                      2
```

Report 3: Rating Distribution

Number of 5-star ratings

```
SQL> SELECT COUNT(*) AS NO_OF_5_STAR_RATINGS FROM REVIEW_RATING WHERE RATING = 5;

NO_OF_5_STAR_RATINGS
--------------------
                   4
```

Number of 4-star ratings

```
SQL> SELECT B.BRAND_NAME AS PRODUCT_NAME, AVG(R.RATING) AS AVERAGE_RATING FROM BRAND B JOIN REVIEW_RATING R ON B.PRODUCT_ID = R.PRODUCT_ID GROUP BY
B.BRAND_NAME HAVING AVG(R.RATING) < 3 ORDER BY AVERAGE_RATING;

PRODUCT_NAME                 AVERAGE_RATING
---------------------------- --------------
Calvin                                    2

SQL> SELECT COUNT(*) AS NO_OF_4_STAR_RATINGS FROM REVIEW_RATING WHERE RATING = 4;

NO_OF_4_STAR_RATINGS
--------------------
                   3
```

Number of low-rated products

```
SQL> SELECT COUNT(DISTINCT PRODUCT_ID) AS NO_OF_LOW_RATED_PRODUCTS FROM REVIEW_RATING WHERE RATING < 3;

NO_OF_LOW_RATED_PRODUCTS
------------------------
                       1
```

Complete Rating

```
SQL> SELECT RATING, COUNT(*) AS NO_OF_RATINGS FROM REVIEW_RATING GROUP BY RATING ORDER BY RATING DESC;

    RATING NO_OF_RATINGS
---------- -------------
         5             4
         4             3
         3             2
         2             1
```